

Das Netzwerk

Handbuch zur Visual Novel

Olav Schettler

Version 1.0, 2026-08-19: Erstentwurf

Table of Contents

Vorwort	1
1. Worum es geht	2
1.1. Die Visual Novel	2
1.2. Woher der Stoff kommt	2
1.3. Was Decker ist	2
1.4. Für wen dieses Handbuch ist	3
2. Das Spiel spielen	4
2.1. Öffnen	4
2.2. Die Titelfarbe	4
2.3. Der Kapitelindex	5
2.4. Der Menü-Knopf	5
2.5. Speichern und Weiterspielen	6
2.6. Wie Entscheidungen wirken	6
3. Einrichtung	8
3.1. Was du installiert brauchst	8
3.2. Das fertige Spiel öffnen	8
3.3. Das Spiel selbst neu bauen	8
3.4. Was der Bau prüft	9
3.5. Das Handbuch selbst bauen	9
4. Software-Architektur	11
4.1. Drei Module, eine Erzählmaschine	11
4.2. Die Engine als Deck-Skript	12
4.3. Warum die ganze Geschichte in einer blockierenden Schleife läuft	12
4.4. Von einer Ply-Passage zu Prosa und Auswahl	13
4.5. Der Spielstand als %J-Zeichenkette	15
4.6. Die Karten des Decks	15
5. Das Datenformat	17
5.1. Passagen	17
5.2. Tags	17
5.3. Kapiteltitel als Metadatum	17
5.4. Bezeichner ohne Umlaute, Text mit Umlauten	18
5.5. Regieanweisungen	18
5.6. Sprecherpräfix	19
5.7. Die ?-Frage	19
5.8. Links: eine Fortsetzung oder eine Entscheidung	19

5.9. Variablen und bedingter Text	20
5.10. Eine vollständige Beispielpassage	21
6. Die Bilder	23
6.1. <code>ink.py</code> : der Strichgenerator	23
6.2. <code>portraits.py</code> : Gesichtsgeometrie je Figur, Emotes als Tabelle	24
6.3. Drei Blickrichtungen, Blinzel- und Sprechbilder	26
6.4. Hintergründe: <code>szenen.py</code>	26
6.5. Von SVG zu GIF: <code>render_art.py</code> und die Silhouetten-Kette	29
7. Die Handlung	32
7.1. Grundsatz: die Novelle bleibt kanonisch	32
7.2. Vier Teile und ein Epilog	32
7.3. Teil II: die Übersichtskarte	35
7.4. Kapitel 15: die Abstimmung	35
7.5. Die Variablen im Überblick	36
8. Bearbeiten und erweitern	38
8.1. Text ändern	38
8.2. Eine Szene ergänzen	38
8.3. Eine Entscheidung einbauen	39
8.4. Ein Emote hinzufügen	39
8.5. Einen Hintergrund ergänzen	40
8.6. Was der Bau prüft, und was die Fehlermeldungen bedeuten	40
8.7. Änderungen automatisch neu bauen	41
9. Lil für Ungeübte	43
9.1. Werte und Variablen	43
9.2. Funktionen aufrufen und definieren	43
9.3. Bedingungen und Schleifen	43
9.4. Die wichtigste Falle: Lil hat keine Rechenregeln	43
9.5. Zehn Fallstricke, alle tatsächlich beim Bau dieses Projekts aufgetreten	44
9.6. Ein echtes Beispiel: Sprechererkennung	46
9.7. Zum Weiterlesen	46
10. Glossar	48
Impressum	51

Vorwort

Dieses Handbuch gehört zur Visual Novel **Das Netzwerk**, einer spielbaren Adaption der gleichnamigen Novelle, gebaut mit [Decker](#) und der Sprache Lil. Es richtet sich an alle, die die Geschichte einfach nur spielen wollen, genauso wie an alle, die selbst eine Szene ändern, eine Entscheidung einbauen oder eine neue Zeichnung ergänzen möchten, auch wenn sie noch nie programmiert haben.

Du brauchst für die ersten Kapitel keine Vorkenntnisse. Wo es technischer wird, sagen wir das vorher an, und du kannst diese Abschnitte auch einfach überspringen, wenn sie dich gerade nicht interessieren.

Chapter 1. Worum es geht

1.1. Die Visual Novel

Das Netzwerk ist eine spielbare Adaption der Novelle "Das Netzwerk": eine lineare, bebilderte Erzählung mit Entscheidungspunkten, kein Adventure und kein Tilemap-Spiel. Du klickst dich durch Dialogboxen, siehst sechs Figuren als Tuschezeichnungs-Puppen auf einer Bühne und triffst an ausgewählten Stellen Entscheidungen, die den Tonfall und den Epilog beeinflussen.

Die Novelle selbst erzählt von sechs Mitarbeitenden eines fiktiven öffentlich-rechtlichen Auslandssenders, dem "Bundesrundfunk International" (BRI), die zwischen Januar 2026 und März 2030 heimlich ein föderiertes, zensurresistentes Sendernetzwerk aufbauen, weil die Zentrale den Journalismus nicht mehr schützen kann. Die Handlung bleibt in der Adaption vollständig erhalten: 30 Kapitel, vier Teile plus Epilog. Was du als Spieler beeinflusst, ist nicht **ob** die Geschichte passiert, sondern **mit welcher Haltung** die Figuren hindurchgehen, siehe [Die Handlung](#).



Die Novelle und ihre Adaption sind vollständig fiktional. Alle Figuren, Institutionen und Ereignisse sind frei erfunden. Der ausführliche rechtliche Hinweis steht im Spiel selbst hinter dem Knopf "Rechtlicher Hinweis" auf der Titelfarte.

1.2. Woher der Stoff kommt

Die Vorlage ist eine 19.445 Wörter lange Prosa-Novelle in AsciiDoc, 30 Kapitel von Januar 2026 bis März 2030. 26 der 30 Kapitel tragen bereits strukturierte Metadaten (Erzählperspektive, Datum, Ort). Aus dieser Prosa wurde die Spielszene: gekürzt, in Dialog aufgelöst, mit Regieanweisungen versehen. Aus rund 19.000 Wörtern Prosa wurden etwa 12.000 Wörter Spieltext mit 32 Entscheidungspunkten. Der komplette Bauplan mit allen Entscheidungen und ihrer Begründung steht in [plans/graphic-novel-plan.md](#), dem Ausgangspunkt fast aller Kapitel dieses Handbuchs.

1.3. Was Decker ist

Decker ist das Programm, mit dem man das fertige Spiel spielt und mit dem man Bilder, Karten und Knöpfe von Hand zusammenklicken kann, so ähnlich wie bei einem Karteikasten (daher der Name). Decker wurde von John Earnest gebaut und ist kostenlos. **Lil** ist die kleine Programmiersprache, die in Decker eingebaut ist. Für "Das Netzwerk" kommen noch drei fertige Decker-Module dazu, die die eigentliche Erzählmaschine bilden:

Modul	Schicht	Liefert
twee (Ply)	Struktur	Passagen, Links, Variablen, eingebettete Lil-Fragmente
dd (Dialogizer)	Präsentation	modale Dialogbox, Klick-durch, Auswahlknöpfe, Textanimation
pt (Puppeteer)	Darstellung	Figuren auf der Bühne, Emotes, Blinzeln, Sprechanimation, Bewegung

Diese Module sind nicht dafür gebaut, zusammen zu laufen, passen aber genau ineinander. Wie das zusammenspielt, ist das Kernthema von [Software-Architektur](#).

Bei "Das Netzwerk" kommt außerdem hinzu: der komplette Erzählinhalt (jede Szene, jede Regieanweisung, jede Entscheidung, jede Bedingung) steht in einer einzigen Textdatei, `game/data/szenen.twee`. Ein Bauprogramm namens `build_vn.lil` liest diese Datei und baut daraus automatisch das fertige Deck. Das ist der Grund, warum du für die meisten Änderungen keine Programmiersprache lernen musst, siehe [Bearbeiten und erweitern](#).

1.4. Für wen dieses Handbuch ist



Wenn du nur spielen willst: lies [Das Spiel spielen](#) und [Einrichtung](#), das reicht komplett. Alles danach ist für alle, die tiefer einsteigen wollen: wie die Engine funktioniert, wie das Datenformat aufgebaut ist, wie die Tuschezeichnungen entstehen, und wie du selbst etwas ergänzt.

[Software-Architektur](#) und [Das Datenformat](#) sind die technischen Kernkapitel und setzen etwas Geduld voraus. [Die Bilder](#) erklärt, wie aus Python-Code Tuschezeichnungen werden. [Die Handlung](#) beschreibt die Geschichte selbst, ihre Struktur und ihre Variablen. [Bearbeiten und erweitern](#) zeigt dir konkrete Arbeitsabläufe zum Weiterschreiben. [Lil für Ungeübte](#) gibt dir gerade so viel Lil, wie du zum Ändern der Engine brauchst, inklusive der Fallstricke, in die beim Bau dieses Projekts tatsächlich jemand getreten ist.

Chapter 2. Das Spiel spielen

2.1. Öffnen

Zwei Wege, "Das Netzwerk" zu starten:

- Decker öffnen, dann über **Datei** → **Öffnen** die Datei `game/build/netzwerk.deck` auswählen.
- Oder `game/build/netzwerk.html` direkt in einem Browser öffnen, ganz ohne Decker zu installieren. Praktisch, wenn du das Spiel jemandem zeigen willst, der Decker nicht hat.

Steuerung ist ausschließlich die Maus: Text weiterklicken, eine Antwortmöglichkeit anklicken. Es gibt keine Tastenkürzel, die du lernen müsstest.



Sobald das Spiel in Decker offen ist, achte darauf, dass das Werkzeug **Interact** ausgewählt ist (oben in der Werkzeugleiste, oder Taste **F1**). Nur dann reagiert das Deck auf Klicks statt auf Bearbeitungswerkzeuge.

2.2. Die Titelfarte



Auf der Titelfarte stehen vier Knöpfe:

Knopf	Wirkung
Neu beginnen	Startet die Geschichte von vorn, ab der ersten Passage der Szenendatei. Ein vorhandener Spielstand wird dabei überschrieben.
Weiterspielen	Erscheint nur, wenn es einen unfertigen Spielstand gibt (siehe unten), und setzt die Geschichte an genau der Stelle fort, an der du zuletzt warst.
Kapitel	Öffnet den Kapitelindex, siehe unten.
Rechtlicher Hinweis	Zeigt den vollständigen rechtlichen Hinweis der Novelle, wortgleich aus <code>game/data/hinweis.txt</code> .

Darunter zeigt die Titelfarte deinen Fortschritt: "Noch nicht begonnen.", "Zuletzt: <Kapitelindex>." oder "Durchgespielt.", je nachdem, was der gespeicherte Spielstand hergibt.

2.3. Der Kapitelindex

Der Kapitelindex listet jeden Kapitelanfang mit seinem Titel. Bereits gespielte Kapitel (und immer das erste) sind anklickbar und springen direkt dorthin; noch nicht erreichte stehen mit dem Zusatz "(noch nicht gespielt)" da und lassen sich nicht anklicken. Das ist keine separate Tabelle, die Liste entsteht aus den Metadaten der Szenendatei selbst, siehe [Das Datenformat](#).

Ein Sprung aus dem Index setzt die Geschichte an diesem Kapitelanfang fort, mit denselben Variablen, die du bis dahin gesammelt hast. Es ist ein Sprung innerhalb derselben Geschichte, kein neuer Anfang.

2.4. Der Menü-Knopf

Auf jeder Karte außer der Titelfarte selbst sitzt oben rechts ein Knopf **Menü**. Ein Klick darauf sichert deinen aktuellen Stand und bringt dich zurück zur Titelfarte.



Technisch ist das ein Sonderfall: während eine Textbox oder eine Auswahl auf dem Bildschirm steht, blockiert Decker die normale Ereigniszustellung, ein gewöhnlicher Klick-Handler auf dem Menü-Knopf würde also nie ausgelöst. Die Engine prüft die Zeigerposition deshalb selbst bei jedem Animationsereignis (`vn_heim_geklickt` in `game/lil/vn.lil`) und bricht die Erzählschleife sauber zwischen zwei Textboxen ab, siehe [Software-Architektur](#).

2.5. Speichern und Weiterspielen

Nach jeder einzelnen Passage schreibt die Engine deinen kompletten Spielstand (aktuelle Passage, alle Variablen, welche Passagen du schon gesehen hast) in ein unsichtbares Feld auf der Karte **daten**. Du musst dafür nichts tun, es passiert automatisch. "Weiterspielen" auf der Titelfarte erscheint deshalb genau dann, wenn dieser Stand existiert und die Geschichte noch nicht zu Ende gespielt wurde.



Wie beim zugehörigen Tilemap-Spiel "Das Netzwerk" (zelda-network) gilt: die interaktive Decker-App darf aus Sicherheitsgründen keine Dateien selbst von der Festplatte nachladen. Nach jedem Neubau des Decks (siehe [Einrichtung](#)) musst du **netzwerk.deck** deshalb in Decker neu öffnen, damit deine Änderungen ankommen. Der gespeicherte Spielstand steckt **im** Deck selbst, ein frisch gebautes Deck hat also wieder einen leeren Stand.

2.6. Wie Entscheidungen wirken

An den meisten Entscheidungspunkten stehen zwei bis drei Antwortmöglichkeiten. Über den Antwortknöpfen steht eine Frage, entweder die Standardfrage "Was tust du?" oder eine eigene, die in der Szenendatei mit einem **?** eingeleitet wird.



Die meisten dieser Entscheidungen ("Haltungswahlen") führen nach ein bis zwei Passagen

wieder in denselben Handlungsstrang zurück, verändern dabei aber eine der drei Haltungsvariablen (**entschlossenheit**, **vorsicht**, **zusammenhalt**). Diese Variablen färben späteren Text (siehe die `{if ... else ... end}`-Stellen in der Szenendatei) und am Ende den Epilog, ändern aber nicht, **welche** Ereignisse stattfinden: die Novelle bleibt in ihrem Verlauf kanonisch. Die einzige echte Verzweigung der gesamten Geschichte liegt in Kapitel 15, der Abstimmung. Mehr dazu, mit allen Variablen im Detail, in [Die Handlung](#).

Chapter 3. Einrichtung

Zum reinen Spielen brauchst du nur Decker, siehe [Das Spiel spielen](#). Alles Weitere auf dieser Seite ist nur nötig, wenn du selbst am Spiel bauen willst.

3.1. Was du installiert brauchst

Werkzeug	Wofür
Decker	Das fertige Deck spielen. Kostenlos für Mac, Windows und Linux.
lilt	Kommandozeilen-Werkzeug, liegt einer normalen Decker-Installation bei (gleicher Ordner wie das Decker-Programm). Baut das Deck und führt die Tests aus.
rsvg-convert	Wandelt die generierten SVG-Zeichnungen in PNG um, der erste Schritt der Bildpipeline, siehe Die Bilder .
ImageMagick (magick)	Schwellwertet die PNGs auf zwei Farben und schneidet sie nach GIF frei, der zweite Schritt der Bildpipeline.
Python 3	Führt tools/portraits.py , tools/szenen.py und tools/render_art.py aus. Kein zusätzliches Paket nötig, nur die Standardbibliothek.
Docker Desktop	Nur um dieses Handbuch selbst zu bauen, siehe unten. Fürs Spiel selbst nicht nötig.

Auf einem Mac mit Homebrew installierst du die Kommandozeilen-Werkzeuge mit:

```
brew install librsvg imagemagick
```

[lilt](#) prüfst du mit `lilt -e '1+1'` in einem Terminal; kommt `2` zurück, ist es gefunden und im Suchpfad.

3.2. Das fertige Spiel öffnen

Im Projekt-Ordner liegt bereits eine fertig gebaute Version unter [game/build/netzwerk.deck](#), plus [game/build/netzwerk.html](#) für den Browser. Siehe [Das Spiel spielen](#) für die Details.

3.3. Das Spiel selbst neu bauen

Wenn du an [game/data/szenen.twee](#), [game/data/emotes.csv](#) oder den Zeichnungen etwas änderst, musst du das Deck neu bauen, damit die Änderung im Spiel ankommt:

```
cd visualnovel-network
lilt game/build/build_vn.lil
```

Das liest `game/data/szenen.twee`, prüft die Daten (siehe unten) und baut daraus `game/build/netzwerk.deck` und `game/build/netzwerk.html` neu. Das dauert nur wenige Sekunden, solange sich an den Zeichnungen nichts geändert hat.

Kürzer geht es mit dem **Makefile** im Projekt-Wurzelverzeichnis:

```
make deck      # entspricht dem lilt-Aufruf oben
make test     # Engine-Tests, game/lil/vn_test.lil
make art      # SVG nach GIF wandeln, mit Histogramm-Pruefung
make portraits # Portraits aus tools/portraits.py neu zeichnen
make alles    # art, test und deck nacheinander
make watch    # baut automatisch neu bei jeder Aenderung, braucht entr
make help     # zeigt alle Ziele
```



`make watch` beobachtet `game/data`, `game/lil` und `game/build` und baut bei jeder Änderung an einer `.twee`-, `.lil`-, `.csv`- oder `.txt`-Datei automatisch neu (braucht das Werkzeug `entr`, `brew install entr`). Das ersetzt aber nicht das erneute Öffnen in Decker: die interaktive App darf aus Sicherheitsgründen keine Dateien selbst von der Platte nachladen, das darf nur `lilt`.

3.4. Was der Bau prüft

`build_vn.lil` liest nicht nur die Szenendatei ein, es prüft sie auch mit, bevor überhaupt ein Deck entsteht: fehlende Sprungziele, unbekannte Puppen, unbekannte Emotes, unbekannte Bühnenpositionen, unbekannte Kommandos, überlappende Puppen und fehlerhafte `{lil}`-Fragmente. Ein fehlender Hintergrund ist nur eine Warnung. Welche Fehlermeldung was genau bedeutet und wie du sie behebst, steht ausführlich in [Bearbeiten und erweitern](#).

3.5. Das Handbuch selbst bauen

Dieses Handbuch (das Dokument, das du gerade liest) ist AsciiDoc-Quelltext unter `handbuch/src/` und wird per Docker zu HTML, EPUB und PDF gebaut, ganz unabhängig vom Spiel selbst:

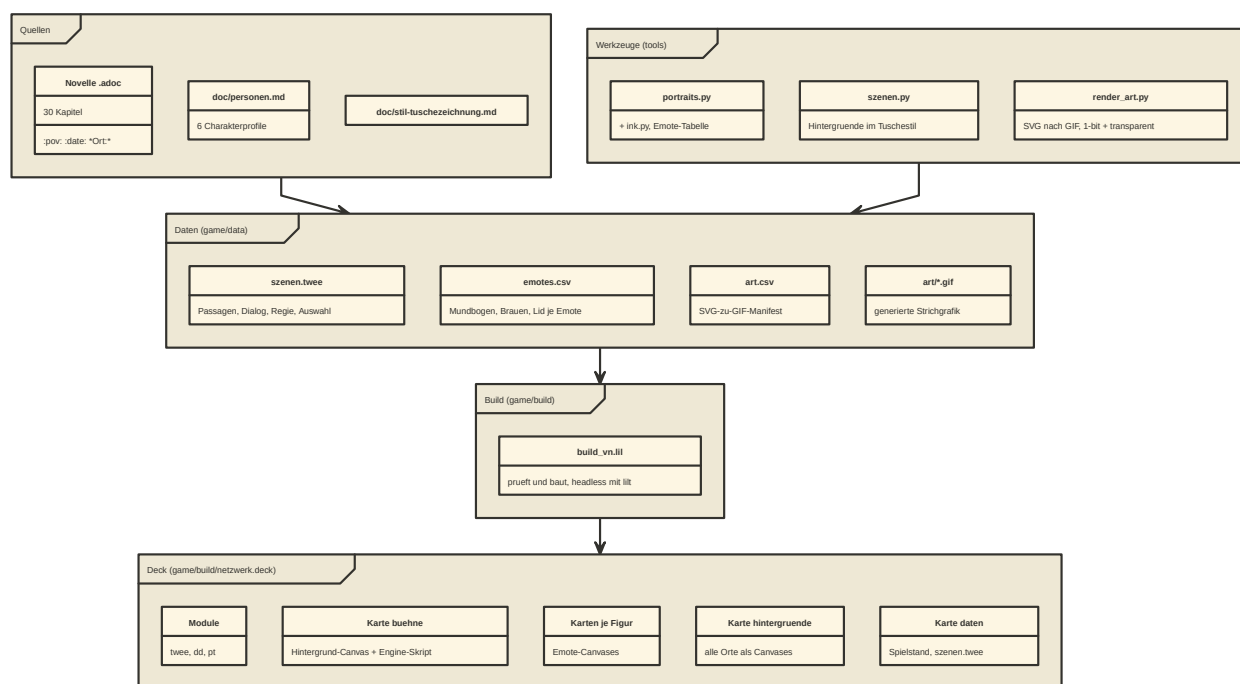
```
cd handbuch/build
make html  # am einfachsten zum Nachschauen
make epub
make pdf
make all   # epub und pdf zusammen
```

Details dazu, inklusive wie du ein Kapitel oder ein Diagramm änderst, stehen in [handbuch/README.md](#).

Chapter 4. Software-Architektur

Dieses Kapitel ist das technische Kernkapitel des Handbuchs. Es erklärt, wie aus drei fertigen, nicht füreinander gebauten Decker-Modulen und einer einzigen Textdatei eine spielbare Erzählung wird. Wenn du nur Text ändern oder Szenen ergänzen willst, brauchst du dieses Kapitel nicht unbedingt, siehe [Bearbeiten und erweitern](#). Wenn du an der Engine selbst etwas ändern willst, ist es Pflichtlektüre.

4.1. Drei Module, eine Erzählmaschine



Die drei Decker-Module decken je eine Schicht ab:

Modul	Schicht	Liefert
twee / Ply	Struktur	Passagen, Links, Variablen, eingebettete Lil-Fragmente
dd (Dialogizer)	Präsentation	modale Dialogbox, Klick-durch, Auswahlknöpfe, Textanimation
pt (Puppeteer)	Darstellung	Figuren auf der Bühne, Emotes, Blinzeln, Sprechanimation, Bewegung

Keines dieser Module wurde entworfen, um mit den anderen beiden zusammenzuarbeiten. Dass sie trotzdem genau ineinanderpassen, wurde vor dem eigentlichen Bau **headless mit lil** **geprüft**, bevor überhaupt eine Zeile Engine-Code entstand ([plans/graphic-novel-](#)

`plan.md`, Abschnitt 2). Zwei Befunde daraus tragen die ganze Architektur:

- `twee.render[]` liefert eine Tabelle mit den Spalten `text`, `font`, `arg` und `pat`. Zeilen mit leerem `arg` sind Prosa, Zeilen mit gefülltem `arg` sind Auswahlmöglichkeiten mit `arg` als Sprungziel. Mehr dazu unten.
- Eine Regieanweisung wie `!show sarah Sorge centerleft` ist für Ply gewöhnlicher Fließtext und landet unverändert in der Prosa. Dialogizer erkennt jede Zeile in Rich Text, die mit `!` beginnt, als Kommando und schickt sie als `command`-Ereignis an die aktuelle Karte. Regie ist damit reiner Text in der twee-Datei, kein Sonderfall im Code.

`build_vn.lil` baut das Deck deshalb aus drei fertigen Modul-Decks zusammen, ganz ohne die Module selbst anzufassen:

```
nd:newdeck[]
nd.add[tweedeck.modules.twee]
nd.add[read["" fuse (DECKS, "dialog.deck")].modules.dd]
nd.add[read["" fuse (DECKS, "puppeteer.deck")].modules.pt]
```

4.2. Die Engine als Deck-Skript

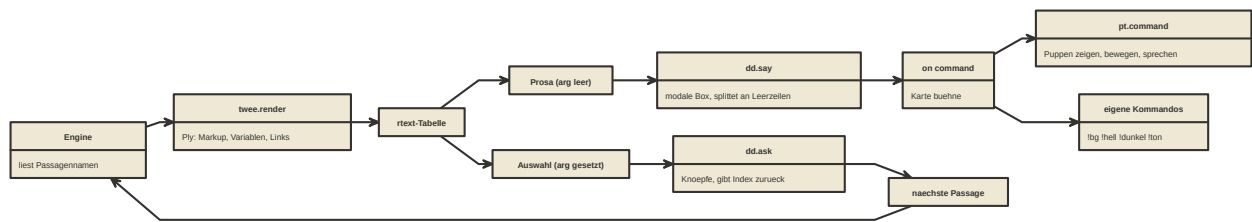
Der eigentliche Klebstoff zwischen den drei Modulen ist `game/lil/vn.lil`, gut 650 Zeilen Lil. `build_vn.lil` setzt diese Datei als Skript des gesamten Decks ein:

```
nd.script:read["game/lil/vn.lil"]
```

Decker verkettet Deck-, Karten- und Widget-Skripte: jede Karte und jeder Knopf im fertigen Deck sieht deshalb alle Funktionen, die in `vn.lil` definiert sind (`vn_spiele`, `vn_kommando`, `vn_sprecher`, und so weiter). Ein Klick auf "Neu beginnen" auf der Titelfarte ruft am Ende nur `vn_start[1]` auf.

4.3. Warum die ganze Geschichte in einer blockierenden Schleife läuft

Das ist die zentrale Entwurfsentscheidung der Engine: es gibt **eine einzige Funktion**, `vn_spiele`, die die komplette Erzählung von der Startpassage bis zum Ende (oder bis zum Abbruch über den Menü-Knopf) durchläuft, in einer `while`-Schleife. Möglich ist das, weil `dd.say[]` und `dd.ask[]` **synchron** sind: sie kehren erst zurück, wenn die spielende Person geklickt hat. Ein kompletter Spieldurchlauf passt damit in einen einzigen Skriptaufruf.



Während `dd.say[]` oder `dd.ask[]` intern warten, schickt Decker trotzdem weiterhin `animate`-Ereignisse an die aktuelle Karte, sodass Puppen weiter blinzeln und Regieanweisungen greifen, ohne dass die Hauptschleife dafür etwas Besonderes tun muss:

```
on animate do
  pt.animate[deck]
  if deck.card~buehne
    if vn_heim_geklickt[] buehne.widgets.abbruch.text:"1" end
  end
end
```

Genau dieser Nebeneffekt macht auch den Menü-Knopf möglich: solange die Hauptschleife läuft, blockiert Decker die normale Klick-Zustellung, aber `animate` kommt weiter durch. `vn_heim_geklickt` liest deshalb Zeigerposition und Klickstatus direkt aus `pointer.pos` und `pointer.down` und setzt nur einen Merker (`buehne.widgets.abbruch`), den die Hauptschleife zwischen zwei Textboxen prüft und dann sauber abbricht.

4.4. Von einer Ply-Passage zu Prosa und Auswahl

Der Kern der Hauptschleife rendert eine Passage und trennt das Ergebnis in zwei Teile:

```
r:twee.render[story s.passage s.vars]
s.vars:r.vars
zeilen:r.value
# ACHTUNG: "where 0<count arg" waere ein stiller No-Op-Filter, weil count
# die ganze Spalte aggregiert statt zeilenweise zu zaehlen. Richtig ist
# die zeilenweise Form "where !arg=\"\"".
prosa:select where arg="" from zeilen
wahl:select where !arg="" from zeilen
```

`twee.render[]` liefert für eine Passage mit Text und zwei Links eine Tabelle mit den Spalten `text`, `font`, `arg` und `pat`. Zeilen, deren `arg`-Spalte leer ist, sind normaler Fließtext oder Regie und gehen an `dd.say[]` bzw. an die eigene Kommandoerkennung. Zeilen mit gefülltem `arg` sind Ply-Links: `text` ist die Beschriftung des Knopfes, `arg` ist der Name der Zielpassage.



Der naheliegende Ausdruck `where 0<count arg` sieht nach derselben Bedingung aus, filtert in Lils Query-Syntax aber überhaupt nicht: `count` aggregiert dort die ganze Spalte zu einer einzigen Zahl statt zeilenweise zu

zählen, die Bedingung ist damit für jede Zeile konstant wahr. In einer frühen Fassung der Engine führte das zu einer Endlosschleife, weil als Sprungziel ein leerer Passagenname herauskam. Gefunden hat das der Test, nicht das Lesen. Die korrekte, zeilenweise Form ist `where !arg=""` (In `where`-Klauseln gehört ohnehin `=` hin, nicht `~`, siehe [Lil für Ungeübte](#)).

Danach unterscheidet die Schleife drei Fälle, je nachdem wie viele Auswahlzeilen es gibt:

```
if 0~count wahl
  laeuft:0
elseif 1~count wahl
  vn_erzaehle[first labels]
  s.passage:first ziele
  vn_speichern[s]
else
  dd.style[vn_stil_frage[]]
  i:dd.ask[frage labels]
  s.passage:ziele[i]
  vn_speichern[s]
end
```

Keine Auswahl heißt: Ende der Erzählung. Genau eine Auswahl ist keine Entscheidung, sondern eine reine Fortsetzung, und wird als letzte Textbox gezeigt statt als sinnloser Ein-Knopf-Dialog (deshalb tragen Fortsetzungslinks in der Szenendatei sprechende Beschriftungen wie "Es klopft." statt Platzhaltern wie "Weiter."). Erst ab zwei Auswahlzeilen erscheint die Frage mit echten Knöpfen über `dd.ask[]`.

Wie eine Passage im Detail zu Prosa und Regie wird, inklusive Sprechererkennung, steht in `vn_prosa`:

```
on vn_prosa zeilen do
  frage:"Was tust du?"
  segmente:rtext.split["\n\n" zeilen]
  each seg in segmente
    roh:rtext.string[seg]
    t:vn_ohne_rand[roh]
    if vn_abbruch[]
      t:""
    elseif "?"~t[0]
      frage:vn_rest[t "?"]
    elseif "!"~t[0]
      command[t]
    else
      if 0<count t vn_sag[seg roh] end
    end
  end
  frage
end
```

Kommandos werden hier absichtlich selbst erkannt, nicht `dd.say[]` überlassen: Dialogizer prüft beim Kommando-Erkennen nur das allererste Zeichen eines Absatzes. Ein Lil-Fragment davor, das nichts ausgibt (etwa `{vorsicht:vorsicht+1 ""}`), hinterlässt einen Zeilenumbruch, und die Regieanweisung dahinter wäre dann stillschweigend zu Fließtext geworden. Deshalb schneidet `vn_ohne_rand` führende Leerräume selbst ab, bevor geprüft wird.

4.5. Der Spielstand als %J-Zeichenkette

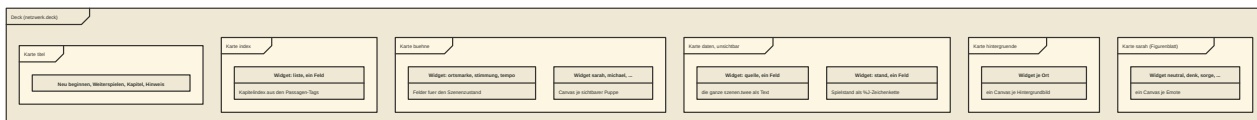
Der komplette Spielstand ist ein einziges Dictionary (`passage`, `vars`, `gesehen`, `beendet`) und wird nach jeder Passage in ein unsichtbares Feld auf der Karte `daten` geschrieben:

```
on vn_speichern s do
  daten.widgets.stand.text:"%J" format s
end

on vn_laden do
  t:daten.widgets.stand.text
  if 0~count t 0 else "%J" parse t end
end
```

`"%J" format` wandelt ein Lil-Dictionary in eine JSON-Zeichenkette, `"%J" parse` liest sie verlustfrei zurück, auch mit Umlauten im Text. Weil dieses Feld ein gewöhnliches Widget des Decks ist, überlebt der Spielstand das Schließen und Wiederöffnen von Decker, obwohl die Erzählschleife selbst niemals unterbrochen wird, sondern nur zwischen zwei Textboxen sauber endet.

4.6. Die Karten des Decks



`build_vn.lil` legt folgende Karten an:

Karte	Inhalt
<code>titel</code>	Die Knöpfe "Neu beginnen", "Weiterspielen", "Kapitel", "Rechtlicher Hinweis" sowie eine Fortschrittsanzeige.
<code>index</code>	Ein Feld mit dem Kapitelindex, aus den Passagen-Metadaten erzeugt.

Karte	Inhalt
hinweis	Der rechtliche Hinweis der Novelle, wortgleich aus <code>game/data/hinweis.txt</code> .
buehne	Die eigentliche Spielkarte: Ortsmarke, verdeckte Zustandsfelder (<code>stimmung</code> , <code>tempo</code> , <code>erzaehlerpos</code> , <code>abbruch</code>) und, sobald eine Figur auftritt, ein Canvas-Widget je sichtbarer Puppe, von Puppeteer selbst angelegt.
daten	Unsichtbar im Spiel. Enthält die komplette <code>szenen.twee</code> als Text (<code>quelle</code>) und den aktuellen Spielstand (<code>stand</code>).
hintergruende	Unsichtbar. Ein Canvas je Ort, aus <code>game/data/art/hintergruende/*.gif</code> .
eine Karte je Figur (<code>sarah</code> , <code>michael</code> , <code>jamal</code> , <code>kat</code> , <code>lena</code> , <code>tom</code> und ihre <code>_r/_l</code> -Varianten)	Puppeteers Figurenblätter: ein Canvas je Emote, siehe Die Bilder .

Hintergründe werden zur Laufzeit nicht als Kartenwechsel behandelt, sondern als Kopie eines Canvas in das Bühnenbild:

```
on vn_hintergrund name do
  h:hintergruende.widgets[name]
  if h
    buehne.image:h.copy[]
  else
    buehne.image.clear[]
  end
end
```

Das vermeidet jedes Nachladen von der Platte, was die interaktive Decker-App aus Sicherheitsgründen ohnehin nicht dürfte.

Chapter 5. Das Datenformat

Die gesamte Geschichte, mit jeder Regieanweisung, jeder Entscheidung und jeder Variable, steht in einer einzigen Datei: `game/data/szenen.twee`, gut 3300 Zeilen, echtes Tweep 3 im Story-Format **Ply**. Damit ist die Datei sowohl in Twine bearbeitbar (Story-Format über <https://beyondloom.com/decker/ply/format.js> hinzufügen) als auch von `twee.read[]` direkt lesbar. Wer eine Szene ändern, eine Figur auftreten lassen, einen Hintergrund wechseln oder eine Entscheidung einbauen will, editiert diese eine Datei und baut neu, siehe [Bearbeiten und erweitern](#).

5.1. Passagen

Eine Passage ist eine Szene an einem Ort. Kapitel mit Ortswechsel werden auf mehrere Passagen aufgeteilt, Kapitel ohne auf eine. Der Passagenname folgt dem Muster `k<NN>-<kurzname>`, zum Beispiel `k01-buero` oder `k15-abstimmung`. Der Name ist ausschließlich der Sprungziel-Bezeichner, er wird nie angezeigt.

Kopf einer Passage aus der echten Szenendatei:

```
:: k01-buero [kapitel-01 teil-1 pov-sarah datum-2026-01] {"titel":"Kapitel 1: Sarah - Die Vision"}
```

Das ist: der Passagenname, in eckigen Klammern die Tags, in geschweiften Klammern ein JSON-Objekt mit beliebigen Metadaten.

5.2. Tags

Tags tragen Metadaten für Werkzeuge und den Kapitelindex, nicht für die Darstellung selbst: `kapitel-01`, `teil-1`, `pov-sarah`, `datum-2026-01`. Die Darstellung (Hintergrund, Stimmung, welche Figur wo steht) steuern stattdessen Kommandos im Fließtext, weil die dann an genau der richtigen Stelle im Text stehen und nicht nur passagenweit gelten.

5.3. Kapiteltitel als Metadatum

Ein Kapitelanfang trägt seinen Titel im JSON-Objekt:

```
{"titel":"Kapitel 1: Sarah - Die Vision"}
```

Die Engine baut daraus den kompletten Kapitelindex (`vn_kapitel` in `game/lil/vn.lil`, siehe [Software-Architektur](#)): sie liest jede Passage mit einem `titel`-Schlüssel im Metadaten-Objekt und merkt sich Name und Titel. Zu welchem Kapitel eine beliebige andere Passage gehört,

ermittelt `vn_kapiteltag` über den `kapitel-NN`-Tag, nicht über die Reihenfolge in der Datei, damit Rücksprünge (etwa die frei wählbare Reihenfolge in Teil II) nichts durcheinanderbringen. Twee 3 erlaubt beliebige Schlüssel in diesem Objekt, und `twee.write[]` gibt sie unverändert zurück, Twine behält die Angabe beim Bearbeiten also bei.

5.4. Bezeichner ohne Umlaute, Text mit Umlauten

Passagennamen, Kommandonamen, Puppen- und Emote-Namen, Hintergrund-Namen und Variablennamen sind ASCII (`k01-buero`, `!rueckblende`, `goerlitz-kinderzimmer`). Das gilt nur für Bezeichner. Jeder Text, den die spielende Person zu sehen bekommt, hat selbstverständlich Umlaute, und `lilt`, `twee.read[]` und `twee.render[]` reichen sie verlustfrei durch.

5.5. Regieanweisungen

Eine Zeile, die mit `!` beginnt, ist für Ply gewöhnlicher Fließtext, für die Engine aber eine Regieanweisung. Kommandos stehen als eigener Absatz, durch Leerzeilen von der Prosa abgetrennt; mehrere Kommandos dürfen innerhalb eines Absatzes mit einfachem Zeilenumbruch gruppiert werden.

Kommando	Wirkung	Herkunft
<code>!bg sarah-buero</code>	Hintergrundbild wechseln	eigenes Kommando
<code>!hell / !dunkel / !rueckblende</code>	Dialogizer-Stil umschalten	eigenes Kommando
<code>!zeit Januar 2026, Bonn</code>	Ort- und Zeitmarke einblenden	eigenes Kommando
<code>!schnell / !langsam / !tempo 3</code>	Textgeschwindigkeit	eigenes Kommando
<code>!erzaehler oben / !erzaehler unten</code>	wo der Erzähltext steht	eigenes Kommando
<code>!show sarah Sorge centerleft</code>	Puppe zeigen, mit Emote und Position	<code>pt</code> (Puppeteer)
<code>!move sarah centerright 45</code>	Puppe bewegen	<code>pt</code>
<code>!hide tom / !clear</code>	Puppe(n) entfernen	<code>pt</code>
<code>!shake 5 5 15</code>	Bildschirm wackeln (Betonung)	<code>pt</code>
<code>!play klopfen</code>	Ton abspielen	<code>pt</code>

Kommando	Wirkung	Herkunft
<code>!wait 30</code>	warten, Animation laufen lassen	pt

Es gibt bewusst kein `!talk`-Kommando mehr: wer spricht, ergibt sich aus dem Sprecherpräfix im Text selbst, siehe unten.

5.6. Sprecherpräfix

Sprecher stehen als kurzer Vorspann vor einem Doppelpunkt im Text:

Sarah: Ich weiß.

Erzähltext bekommt kein Präfix. Erkannt wird ein Sprecher an einem Vorspann, der wie ein Name aussieht: höchstens drei Wörter, höchstens 24 Zeichen, jedes Wort großgeschrieben. So bleibt `Dr. Hartmann: ...` ein Sprecher, während `Sie las: der Bescheid war da.` Erzähltext bleibt, weil "las" klein anfängt. Das ist die **einzige** Quelle dafür, wer spricht:

- Die Puppe mit dem kleingeschriebenen Vornamen bewegt den Mund (`Sarah:` steuert die Puppe `sarah`). Gibt es keine solche Puppe, ist es eine Nebenfigur ohne Portrait, und keine Puppe bewegt sich.
- Erzähltext bewegt keine Münder.
- Rede und Erzähltext sehen verschieden aus: Rede folgt der Stimmung der Szene und steht aufrecht, Erzähltext ist immer schwarz auf weiß und steht kursiv. Beide sind linksbündig.
- Zwei Sprecher unterscheiden sich automatisch über die Ausrichtung der Dialogbox: wer links auf der Bühne steht, spricht linksbündig, wer rechts steht, rechtsbündig, abgelesen an der Bühnenposition des zugehörigen Canvas-Widgets.

5.7. Die `?`-Frage

Eine Prosazeile, die mit `?` beginnt, ist die Frage, die über den Auswahlknöpfen erscheint:

? Was tut Sarah mit dem, was sie gerade gelesen hat?

Fehlt sie, nimmt die Engine den Standardtext "Was tust du?".

5.8. Links: eine Fortsetzung oder eine Entscheidung

Auswahl entsteht über Ply-Links am Ende einer Passage:

```
[[Ein Konzept schreiben.->k01-konzept]]
[[Erst das Fenster schließen und nachdenken.->k01-fenster]]
```

Genau ein Link ist keine Auswahl, sondern eine Fortsetzung. Die Engine zeigt dann die Beschriftung als letzte Textbox und springt weiter, statt einen Knopf mit nur einer Möglichkeit anzubieten:

```
[[Es klopft.->k01-michael]]
```

Beschriftungen von Fortsetzungslinks sollten deshalb sprechend sein ("Es klopft.", "Zwei Monate später."), keine Platzhalter wie "Weiter.". Erst ab zwei Links erscheint die Frage mit echten Knöpfen.



Ein Bindestrich in der **Beschriftung** eines Links löscht den Link spurlos: statt eines Auswahlknopfes erscheint der rohe Text `[[Die Chaos-Leute verstehen OpSec.->k06-vertrauen]]` im Spiel. Im **Sprungziel** ist ein Bindestrich dagegen harmlos, und Passagennamen benutzen ihn ständig (`k01-buero`). Nachgemessen mit Minimalbeispielen:

Passage	Ergebnis
<code>[[Chaosleute->x]]</code>	1 Link
<code>[[Chaos-Leute->x]]</code>	0 Links
<code>[[Erster->k06-vertrauen]]</code>	1 Link

`build_vn.lil` bricht deshalb ab, wenn eine Link-Beschriftung einen Bindestrich enthält. Beschriftungen also umformulieren: aus "Wir nennen es ein Resilienz-Projekt." wird "Wir nennen es ein Projekt für Resilienz."

5.9. Variablen und bedingter Text

Variablen werden über `{lil}`-Fragmente gesetzt. Sie bleiben über Passagen hinweg erhalten (`twee.render[]` gibt `r.vars` zurück, die Engine reicht es an den nächsten Aufruf weiter) und werden nach jeder Passage gespeichert:

```
{entschlossenheit:entschlossenheit+1 ""}
```

Das abschließende `""` ist kein Stilmittel, sondern notwendig: ein `{...}`-Fragment, das nur eine Variable setzen soll, muss mit `""` enden, sonst erscheint der Wert der Zuweisung selbst als Text im Spiel.

Bedingter Text entsteht genauso als Fragment, das einen Wert statt eines leeren Strings zurückgibt:

```
{if entschlossenheit>0 "Michael lehnte sich zurück und las, was sie  
geschrieben hatte. Er schwieg lange." else "Michael lehnte sich zurück.  
Sie hatte ihm noch nichts zu zeigen, nur einen Gedanken." end}
```



Ply führt `{...}`-Fragmente aus und schluckt Fehler dabei **stillschweigend**. Ein `if` ohne `end` rendert einfach zu nichts: die bedingte Textstelle fehlt im Spiel, ohne Meldung, ohne Lücke, ohne Hinweis. `build_vn.lil` schneidet deshalb jedes Fragment mit einem Klammer-Tiefenzähler heraus und wertet es einzeln mit `eval[]` aus, das Syntaxfehler im Feld `error` meldet statt sie zu verschlucken, und bricht den Bau bei einem Fehler ab.

5.10. Eine vollständige Beispielpassage

Der Anfang von Kapitel 1, wörtlich aus `game/data/szenen.twee`:

```
:: k01-buero [kapitel-01 teil-1 pov-sarah datum-2026-01] {"titel":"Kapitel 1:  
Sarah - Die Vision"}  
!clear  
!bg sarah-buero  
!zeit Januar 2026, Bonn  
!dunkel
```

Der Schnee fiel leise auf die Dächer der Bonner Innenstadt, als Sarah Hoffmann das Fenster ihres Büros im siebten Stock öffnete. Die kalte Luft brannte in ihren Lungen, aber sie brauchte sie.

```
!show sarah neutral center
```

USAGM workforce reduced by 85 percent. Voice of America effectively silenced.

```
!show sarah schock
```

Fünfundachtzig Prozent. In sechs Monaten. Das war keine Umstrukturierung. Das war eine Hinrichtung.

```
!show sarah denk
```

Ihr Blick fiel auf den kleinen Metallanhänger an ihrem Schlüsselbund. Ein Miniatur-Sendemast, kaum drei Zentimeter hoch.

Sarah: Damit du nie vergisst, wofür wir senden.

Damals hatte sie gelacht. Heute fühlte sich der kleine Mast an wie ein Relikt aus einer anderen Zeit.

? Was tut Sarah mit dem, was sie gerade gelesen hat?


```
[[Ein Konzept schreiben.->k01-konzept]]  
[[Erst das Fenster schließen und nachdenken.->k01-fenster]]
```

```
:: k01-konzept [kapitel-01 teil-1 pov-sarah]  
{entschlossenheit:entschlossenheit+1 ""}  
!show sarah ent
```

Sie kehrte zu ihrem Schreibtisch zurück und öffnete ein neues Dokument.

Sarah: Konzept: Föderierte Medieninfrastruktur für den Bundesrundfunk International.

Sie schrieb zwei Stunden lang. Ein Netzwerk von Hubs, verteilt über die Welt. Jeder Hub autonom, aber verbunden.

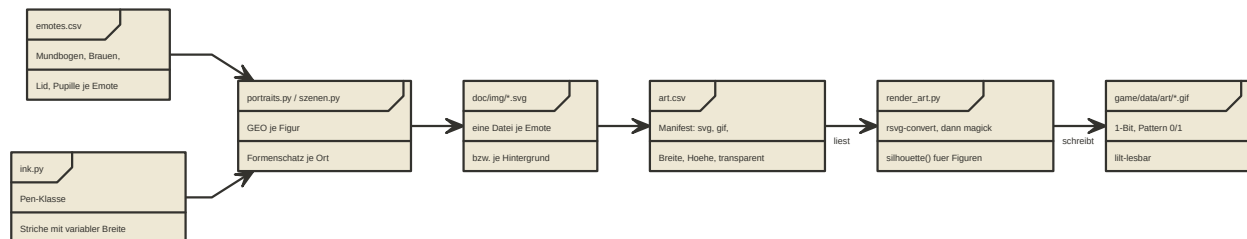
Wenn Berlin das Budget kürzte, würden die Hubs weitersenden. Wenn Bonn die Zentrale schloss, würde das Netzwerk weiterleben.

```
[[Es klopft.->k01-michael]]
```

An dieser Passage lässt sich fast das ganze Format ablesen: **!clear** räumt die Bühne, **!bg** setzt den Hintergrund, **!zeit** die Ortsmarke, **!dunkel** die Stimmung. **!show sarah neutral center** bringt die Puppe auf die Bühne, spätere **!show sarah <emote>**-Zeilen ohne Position wechseln nur den Ausdruck und behalten die zuletzt gültige Position bei (siehe **vn_aktive** in [Software-Architektur](#)). Der Absatz ohne Sprecherpräfix ("USAGM workforce...") ist Erzähltext (in diesem Fall ein wörtliches Zitat), **Sarah: ...** ist Rede. Die **?**-Frage und die zwei Links am Ende von **k01-buero** bilden die erste Entscheidung der Geschichte; die Antwort **k01-konzept** erhöht **entschlossenheit** und führt über einen Ein-Link-Übergang ("Es klopft.") weiter zu **k01-michael**.

Chapter 6. Die Bilder

Jede Zeichnung im Spiel ist generiert, keine ist von Hand in einem Zeichenprogramm entstanden. Das gilt für die sechs Figuren genauso wie für die 18 Hintergründe. Vorlage ist der Tuschezeichnungsstil der Urmel-Bücher (Franz Josef Tripp), verbindlich beschrieben in [doc/stil-tuschezeichnung.md](#). Dieses Kapitel erklärt, wie aus Python-Code Tuschezeichnungen werden, und wie die daraus entstehenden SVGs zu den 1-Bit-GIFs werden, die `lilt` tatsächlich lesen kann.



6.1. `ink.py`: der Strichgenerator

Das zentrale Stilmerkmal ist die **variable Strichbreite**: ein echter Federstrich setzt haarfein an, schwillt an, wo Druck auf der Feder liegt, und läuft spitz aus. Ein SVG-`stroke` mit fester `stroke-width` kann das grundsätzlich nicht, deshalb zeichnet `tools/ink.py` jeden Strich als **gefüllten Umriss** entlang einer Mittellinie mit einem Breitenprofil:

```
def _profile(t, w):
    """Breite an Position t. w = (Anfang, Mitte, Ende).

    Lagrange durch (0,w0), (0.5,w1), (1,w2): die Mittenbreite wird
    tatsaechlich erreicht -- bei quadratischer Mischung waere sie nur halb
    so gross wie angegeben."""
    w0, w1, w2 = w
    return (w0 * 2 * (t - 0.5) * (t - 1)
            - w1 * 4 * t * (t - 1)
            + w2 * 2 * t * (t - 0.5))
```

`Pen.stroke()` nimmt eine Liste von Stützpunkten, glättet sie mit einem Catmull-Rom-Spline (`catmull()`), tastet sie in gleichmäßigen Abständen ab (`resample()`, wichtig, damit das Breitenprofil sauber sitzt) und legt zusätzlich ein niederfrequentes Zittern quer zur Laufrichtung darüber (`wobble()`)—"die Hand ist keine CNC-Fräse", wie der Kommentar im Code sagt. `outline()` baut daraus den geschlossenen Pfad mit runden Endkappen. Auf dieser Basis bietet `Pen` eine kleine Zeichen-API:

Methode	Wofür
<code>stroke()</code>	ein einzelner Federstrich, optional mit <code>cuts</code> für abgesetzte, offene Konturen
<code>blob()</code> / <code>dot()</code>	unregelmäßiger Tuscheklecks bzw. Punkt
<code>patch()</code>	geschlossene Tuschefläche mit unruhigem Rand, für Haarmassen
<code>strands()</code>	eine Haarpartie als Bündel einzelner Strähnen zwischen zwei Leitkurven, die an beiden Enden ausfranst statt als geschlossene Kappe zu stehen
<code>curls()</code>	Lockenkopf aus offenen Spiralzügen
<code>hairs()</code>	Bündel auslaufender Striche, dick am Ansatz, spitz am Ende
<code>spatter()</code>	Tuschespritzer rund um die Figur, mit eigenem Zufallsstrom (siehe unten)
<code>hatch()</code>	lockere, leicht divergierende Schraffur

Auch die Kopfdrehung für die drei Blickrichtungen steckt in `ink.py`, als Zylinderprojektion um die Hochachse (`Pen.set_turn`): Punkte auf der Kopfrundung wandern über die Silhouette hinweg, während deren Rand fast stehen bleibt, genau wie bei einem wirklich gedrehten Kopf. `!flip` wird dafür bewusst **nicht** benutzt, eine gespiegelte Tuschezeichnung sähe falsch aus (der Nasenhaken zeigt dann in die falsche Richtung).



`spatter()` bekommt einen eigenen Zufallsstrom (`rng_seed`), unabhängig vom Rest der Zeichnung. Ohne diese Trennung würden die Tuschespritzer davon abhängen, wie viel Zufall das Gesicht davor schon verbraucht hat, und beim Umschalten zwischen Emotes sichtbar wackeln, obwohl sie auf dem Papier liegen und sich eigentlich nicht bewegen dürften.

6.2. `portraits.py`: Gesichtsgeometrie je Figur, Emotes als Tabelle

`tools/portraits.py` zeichnet die sechs Figuren. Haare, Kleidung, Silhouette und Tuschespritzer sind figurespezifischer Code (die Funktionen `sarah()`, `michael()`, `jamal()`, `kat()`, `lena()`, `tom()`). Augen, Brauen, Nase und Mund dagegen sind **Daten**: jede Figur deklariert ihre Gesichtsgeometrie als Dictionary in `GE0`, zum Beispiel Sarah:

```
"sarah": {
    "auge_l": dict(x=170, y=224, r=7.6, tilt=0.8),
    "auge_r": dict(x=234, y=220, r=7.2, tilt=-0.4),
    "braue_l": dict(pts=[(150, 200), (172, 193), (190, 199)],
                    w=(1.5, 4.6, 0), amp=0.6, aussen_erst=True),
```

```

"braue_r": dict(pts=[(212, 197), (232, 191), (252, 199)],
                 w=(0, 4.8, 1.5), amp=0.6, aussen_erst=False),
"nase": dict(x=202, y0=230, y1=274, w=8),
"mund": dict(x0=181, y0=310, x1=222, y1=306, bow=3),
},

```

Wie ein **Emote** diese Geometrie verändert (Mundbogen, Brauenneigung, Lidstellung, Pupillenversatz, Mundöffnung, Augengröße), steht als Tabelle in `game/data/emotes.csv`, nicht im Python-Code:

```

emote,bow,braue_innen,braue_aussen,lid,pupille_x,pupille_y,mund_offen,augen_gros
s,bogen_daempfung
neutral,3,0,0,1,0,0,0,1.0,1.0
denk,-2,1,-4,1,4,-1,0,1.0,0.6
sorge,-5,-4,2,1,0,1,0,1.0,0.2
ent,0.5,3,3,1,0,0,0,1.0,0.15
freude,9,-1,-2,1,0,0,0,1.0,0.6
schock,-1,-6,-6,0,0,-1,1,1.45,0.3

```

`braue()` verschiebt das innere und das äußere Ende eines Brauenstrichs gemäß `braue_innen` / `braue_aussen`, die Punkte dazwischen linear interpoliert. `mundzug()` baut den Mund aus den Eckpunkten der Figur und einem Bogen, der aus figureneigener Grundkrümmung und Emote-Abweichung gemeinsam entsteht:

```

def wirksamer_bogen(spec, row):
    """Mundbogen aus figureneigener Grundkrümmung und Emote-Abweichung.

    Ohne die Grundkrümmung waeren alle sechs Figuren im neutralen Ausdruck
    gleich, und Kat, Lena und Tom haetten ihren Gesichtsausdruck verloren.
    Wie stark sie in ein Emote hineinwirkt, steht als "bogen_daempfung" in
    der Emote-Tabelle: ausdrucksstarke Emotes wie "ent" verdraengen sie fast
    ganz (sonst grinst Tom auch entschlossen noch), weiche wie "freude"
    behalten sie."""
    neutral_bow = EMOTES["neutral"]["bow"]
    basis = spec.get("bow", neutral_bow)
    if row.get("emote") == "neutral":
        return basis
    return row["bow"] + row["bogen_daempfung"] * (basis - neutral_bow)

```

Diese eine Funktion löst ein Problem, das erst im Bildvergleich auffiel: eine gemeinsame Emote-Tabelle für alle sechs Figuren hätte Kats warmes Lächeln (Grundbogen 15) und Toms Dauergrinsen (21) verschwinden lassen, weil `neutral` für alle denselben Wert setzt. Die Dämpfung sorgt dafür, dass Tom auch besorgt noch etwas freundlicher wirkt als Lena, ohne dass `sorge` bei ihm zum Lächeln wird.

6.3. Drei Blickrichtungen, Blinzel- und Sprechbilder



Jede Figur liegt in drei Ansichten vor: frontal, `_r` (Kopfdrehung +0.30, Gesicht nach rechts, die Puppe steht also links auf der Bühne) und `_l` (Drehung -0.30, steht rechts). Sprechen zwei Figuren miteinander, wenden sie sich dadurch einander zu. Technisch sind das drei eigenständige Puppen mit eigenen Zeichnungen, weil Puppeteer eine Puppe über den Namen ihrer Karte anspricht; in der Szenendatei steht trotzdem immer nur der schlichte Name, die Engine wählt die passende Variante zur Laufzeit aus der Bühnenposition, siehe [Software-Architektur](#).

Pro Figur und Blickrichtung entstehen 14 Canvases:



- sechs Basis-Emotes (`neutral`, `denk`, `sorge`, `ent`, `freude`, `schock`)
- dieselben sechs mit offenem Mund (`<emote>_open`), für die Sprechanimation
- zwei Blinzelbilder (`neutral_blink`, `ent_blink`) für die beiden Emotes, in denen Figuren laut Plan am längsten stehen

`_blink` ist dabei keine eigene Tabellenzeile in `emotes.csv`, sondern ein Schalter in der Zeichenfunktion selbst: das Auge wird durch einen einzelnen geschlossenen Bogen statt Mandel plus Pupille ersetzt. `_open` ist einfach die Emote-Zeile mit `mund_offen=1`. Macht man die Rechnung über alle sechs Figuren und drei Blickrichtungen, kommt man auf 6 Figuren mal 14 Bilder mal 3 Richtungen, also 252 Einzelzeichnungen, generiert in wenigen Sekunden.

6.4. Hintergründe: `szenen.py`

18 Hintergründe decken alle 30 Kapitel ab. `tools/szenen.py` baut sie aus demselben `ink.Pen` wie die Portraits, aber mit einem eigenen **Formenschatz**, abgeleitet aus den Urmel-

Vorlagen: Kasten, Tisch, Stuhl, Fenster, Regal, Serverschrank, Apparat, Kolben, Pflanze, Papierstapel, Mikrophon, Sofa. Drei Grundregeln aus den Vorlagen bestimmen jede Szene:

1. Es gibt keinen Horizont und keine Wände. Die Gegenstände stehen auf weißem Papier.
2. Der Boden ist nur angedeutet: lockere waagerechte Schraffur, ein paar Steinchen, Tuschespritzer, dazu ein schwerer Zug, der die Linie trägt. Nie eine gefüllte Fläche.
3. Geräte und Möbel sind offene, leicht schiefe Umrisse mit wenigen Details: Knöpfe als Kreise, Nieten und Tasten als Punktreihen, Holzplatten als drei parallele Striche.

Ein Kasten zum Beispiel:

```
def kasten(p, x, y, b, h, w=FEIN, offen=True):  
    """Leicht schiefer Kasten.  
  
    Nur eine Kante bricht auf, und die Linien schiessen ueber die Ecken  
    hinaus. Braechen alle vier Kanten symmetrisch in der Mitte, entstunden  
    vier Eckwinkel statt einer Form."""
```

Warum die Bildmitte frei bleibt: die Puppen im Spiel sind transparente Strichgrafik, der Hintergrund scheint also durch sie hindurch, sobald sie nicht selbst eine deckende Fläche bekommen (dazu gleich mehr). Alles, was in der Bildmitte liegt, erschiene sonst mitten im Gesicht der Figuren. Deshalb stehen die Gegenstände in `szenen.py` konsequent am linken und rechten Rand (`LINKS, RECHTS = 150, 874` bei 1024 Pixel Breite), und die Mitte bleibt Papier. Das ist keine Notlösung, sondern genau der Aufbau der Urmel-Vorlagen selbst.





Zwei Fallen, die beim Zeichnen der Formen immer wieder auftauchen:

- Bricht ein Kasten an allen vier Kanten symmetrisch in der Mitte auf, entstehen vier Eckwinkel statt einer Form. Nur eine Kante darf brechen, und die Linien müssen über die Ecken hinauschießen.
- Waagerechte Striche übereinander sind kein Papierstapel, sondern Schraffur. Erst die seitlichen Kanten und ein schwerer Zug oben machen daraus einen Stapel. Dasselbe gilt für Schatten: wenige kurze Striche dicht unter dem Gegenstand, nie ein Block daneben.

6.5. Von SVG zu GIF: `render_art.py` und die Silhouetten-Kette

`lilt` (und damit `Decker`) liest keine `.svg`- oder `.png`-Dateien, nur `.gif`. `tools/render_art.py` liest dafür `game/data/art.csv`, ein Manifest mit den Spalten `svg`, `gif`, `breite`, `hoehe`, `transparent`, und führt für jede Zeile dieselbe zweistufige Kette aus: zuerst `rsvg-convert`, dann `magick`.

```
lauf = subprocess.run(
    ["rsvg-convert", "-w", breite, "-h", hoehe, svg_pfad, "-o", png_pfad],
    capture_output=True, text=True,
)
...
lauf = subprocess.run(
    ["magick", png_pfad, "-background", "white", "-alpha", "remove",
     "-alpha", "off", "-threshold", "60%", sw_pfad],
```



```
)
capture_output=True, text=True,
```



Die Reihenfolge ist entscheidend: erst weiß unterlegen und schwellwerten, dann erst weiß transparent schalten. Der naheliegende direkte Weg über `-background none` erzeugt Anti-Aliasing-Farben an den Kanten (Decker zeigt sie als eigene Muster, zum Beispiel 32, 44, 45, 46) und macht dabei ausgerechnet Schwarz mit transparent — also das Gegenteil dessen, was man will. Das Ergebnis der richtigen Reihenfolge sind exakt zwei Muster: Muster 0 (transparent) und Muster 1 (schwarze Tusche).

Für Hintergründe (`transparent=0` im Manifest) ist damit schon Schluss, das Bild bleibt einfach deckend schwarz-weiß. Für Figuren (`transparent=1`) kommt ein Schritt dazu, und der beginnt schon in der Zeichnung.

6.5.1. Warum die Figur weißes Papier braucht

Eine reine Strichzeichnung besteht nur aus Tusche und Transparenz. Ohne Gegenmaßnahme scheint der Hintergrund mitten durch die Figur, das Fensterkreuz landet im Auge, die Bodenschraffur quer über dem Mund. In den Vorlagen ist die Figur dagegen weißes Papier und verdeckt, was hinter ihr liegt.

Hergestellt wird das nicht durch Bildbearbeitung, sondern beim Zeichnen: `umriss()` in `tools/portraits.py` legt vor allem anderen eine geschlossene Fläche über Haar, Kopf, Hals und Schultern.

```
def umriss(p, kurz):
    """Weisse Silhouette der Figur, vor der Tusche gezeichnet."""
    u = UMRIS[kurz]
    cx = 200
    pts = [(cx - u["hw_seite"], u["seite"])]
    pts += [(cx - u["hw_oben"], u["oben"] + 26), (cx, u["oben"]),
            (cx + u["hw_oben"], u["oben"] + 26)]
    pts += [(cx + u["hw_seite"], u["seite"]),
            (cx + u["hw_haar"], u["haar_y"])]
    pts += SCHULTER_R
    pts += [(400, 524), (400, 528), (0, 528), (0, 524)]
    pts += list(reversed(SCHULTER_L))
    pts += [(cx - u["hw_haar"], u["haar_y"])]
    p.flaeche(pts, wob=1.6)
```

Je Figur sind das sechs Zahlen in `UMRISS`: der Scheitelpunkt, die halbe Breite oben und auf Ohrhöhe, und wo die Haarpartie endet. Die Schulterlinie ist für alle dieselbe und stammt aus `hals_schultern()`.

`Pen.flaeche()` in `tools/ink.py` hängt den geschlossenen Pfad an eine eigene Liste, und `Pen.svg()` gibt ihn vor der Tusche aus, weiß gefüllt. Dazu darf `svg()` das weiße Rechteck über die ganze Fläche weglassen:

```
def svg(self, w=400, h=520, title=None, grund=True):
    ...
    if grund:
        head += f'<rect width="{w}" height="{h}" fill="#fff"/>'
        flaechen = "".join(f'<path d="{d}" />' for d in self.flaechen)
        body = "".join(f'<path d="{d}" />' for d in self.paths)
        return (head
                + f'<g fill="#fff" fill-rule="nonzero">{flaechen}</g>'
                + f'<g fill="#111" fill-rule="nonzero">{body}</g></svg>\n')
```

Die Puppen werden mit `grund=False` geschrieben. Im gerenderten SVG ist damit genau die Figur undurchsichtig und alles daneben durchsichtig, und `silhouette()` in `render_art.py` muss nur noch den Alphakanal als Maske nehmen:

```
schritte = [
    # Alphakanal des gerenderten SVG als Maske: weiss, wo die Figur ist.
    ["magick", png_pfad, "-alpha", "extract", "-threshold", "40%", maske],
    ["magick", sw_pfad, maske, "-alpha", "off",
     "-compose", "CopyOpacity", "-composite", f"GIF:{gif_pfad}"],
]
```



Ein erster Versuch hatte die Silhouette nachträglich aus dem fertigen Bild gewonnen: die Tusche morphologisch schließen, von den Bildecken aus mit einer Signalfarbe fluten, das Nichterreichte als Inneres nehmen. Das versiegelte zwar den Kopf, nicht aber den Rumpf, denn der endet am unteren Bildrand und ist dort offen, sodass die Flut von unten hereinlief. Auch ein größerer Schließradius half nicht. Aus der Zeichnung heraus ist es sowohl einfacher als auch vollständig: vier Bildbearbeitungsschritte sind zu zweien geworden, und die Schultern decken jetzt ebenfalls.

`render_art.py --pruefen` (auch als `make art` im Projekt-Wurzelverzeichnis) ruft nach dem Umwandeln zusätzlich `lilt` auf jede erzeugte Datei auf und prüft das Pixel-Histogramm: bei `transparent=1` dürfen nur die Muster 0, 1 und 32 vorkommen, bei `transparent=0` darf 0 nicht vorkommen. So fällt ein Fehler in der Kette schon beim Bau auf, nicht erst im Spiel als durchscheinendes Gesicht.

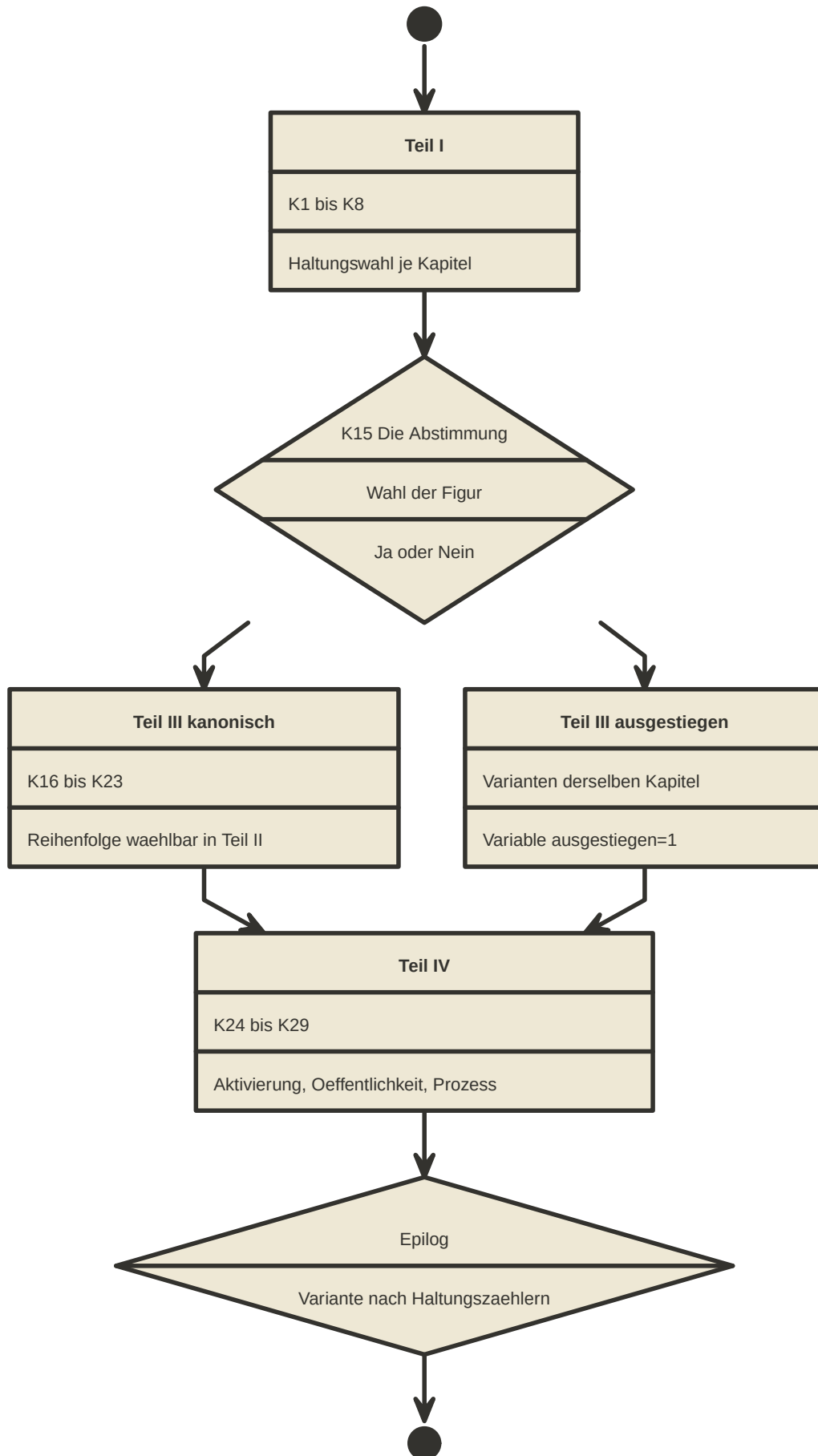
Chapter 7. Die Handlung

7.1. Grundsatz: die Novelle bleibt kanonisch

Die Novelle bleibt in ihrem Verlauf verbindlich: das Team stimmt zu, das Netzwerk entsteht, die Kürzung kommt, das Netzwerk hält. Der Spieler kann diesen Verlauf nicht kippen, das wäre eine andere Geschichte, keine Adaption. Was der Spieler stattdessen tut: **er entscheidet, mit welcher Haltung die Figuren durch die Geschichte gehen.** Genau das ist auch der eigentliche Stoff der Vorlage, und es passt zu einem Medium, das aus Dialogboxen besteht.



7.2. Vier Teile und ein Epilog



30 Kapitel, von Januar 2026 bis März 2030, sechs wechselnde Erzählperspektiven (POV), keine erfundene siebte Spielerfigur:

Kapitel	POV	Zeit	Ort / Hintergrund	Rolle in der Geschichte
1	Sarah	Jan 2026	sarah-buero	Einstieg, erste Haltungswahl
2	Jamal	Mrz 2026	innovation-lab	Figureneinführung
3	Michael	Apr 2026	michael-buero	Figureneinführung
4	Kat	Jun 2026	kats-wg, bri-akademie	Figureneinführung
5	Lena	Sep 2026	lena-buero, cafe-bonn	zwei Passagen (Ortswechsel)
6	Tom	Dez 2026	innovation-lab, ccc-halle, konferenzraum	drei Passagen
7	Sarah	Mrz 2027	konferenzraum	Projektablehnung, Wendepunkt
8	Michael	Jun 2027	konferenzraum	Übergang zu Teil II
9 bis 14	je Figur	Sep 2027	Wohnungen, Büros, Görlitz	sechs Vorbereitungsszenen, Reihenfolge wählbar
15	Alle	16.09.2027	sarahs-wohnung + videokonferenz	Die Abstimmung , echte Verzweigung
16	Sarah	Okt 2027	sarah-buero	erste gefälschte Unterschrift
17	Jamal	Jan 2028	innovation-lab	Architektur
18	Kat	Apr 2028	nairobi-hub	Hub-Aufbau
19	Michael	Jul 2028	michael-buero	Zweifel
20	Lena	Okt 2028	lena-buero	Beinahe-Entdeckung
21	Tom	Jan 2029	innovation-lab	Toms Fehler , Wahl: gestehen oder schweigen
22	Sarah	Apr 2029	sarah-buero	Konfrontation
23	Michael	Jun 2029	michael-buero	Entdeckung durch Dr. Hartmann

Kapitel	POV	Zeit	Ort / Hintergrund	Rolle in der Geschichte
24	Sarah	Jul 2029	sarahs-wohnung	Aktivierung, Höhepunkt
25	Jamal	Aug 2029	innovation-lab	Das Netzwerk lebt
26	Kat	Sep 2029	nairobi-hub	Hubs senden
27	Lena	Okt 2029	pressekonzferenz	Öffentlichkeit
28	Tom	Nov 2029	landgericht	Konsequenzen
29	Michael	Dez 2029	bad-godesberg	Vermächtnis
30	Alle	Mrz 2030	bad-godesberg, BRI	Epilog, Variante nach Variablen

Teil I (Kapitel 1 bis 8) und Teil IV (Kapitel 24 bis 29) sind streng linear, je ein Kapitel nach dem anderen. Teil III (Kapitel 16 bis 23) ist ebenfalls linear, in Abständen von etwa drei Monaten: die Kausalität lässt keine andere Reihenfolge zu, Toms Fehler in Kapitel 21 (Januar 2029) muss vor seiner Entdeckung stehen, nicht danach.

7.3. Teil II: die Übersichtskarte

Nur Teil II hat eine frei wählbare Reihenfolge, weil seine sechs Kapitel tatsächlich am selben Abend spielen: jede der sechs Figuren bereitet sich für sich auf die Abstimmung vor. Umgesetzt ist das über eine Übersichtspassage, **t2-runde**, mit bedingten Links:

```
{if g09 "" else rtext.make["Sarah schreibt die Einladung." "" "k09-einladung"]
end}
{gesamt:g09+g10+g11+g12+g13+g14 if 6~gesamt rtext.make["Der 16. September, 20
Uhr." "" "k15-abstimmung"] else "" end}
```

Jedes der sechs Kapitel setzt beim Verlassen ein eigenes Flag (**g09** bis **g14**) und kehrt zur Übersicht zurück. Die Übersicht zeigt dabei nur noch die Kapitel, deren Flag noch nicht gesetzt ist, als Auswahlmöglichkeit an. `rtext.make[text schrift ziel]` erzeugt dafür eine Zeile mit Sprungziel, die die Engine genau wie einen gewöhnlichen `[[...]]`-Link behandelt. Erst wenn alle sechs Flags stehen (**gesamt** erreicht 6), erscheint überhaupt der Weg zur Abstimmung selbst.

7.4. Kapitel 15: die Abstimmung

Kapitel 15 ist die **einzige echte Verzweigung** der gesamten Geschichte. Der Spieler wählt zunächst, für welche der sechs Figuren er den Zettel schreibt (Variable **figur**):

? Wessen Zettel ist deiner?

```
[[Sarah. Die es angefangen hat.->k15-als-sarah]]  
[[Jamal. Der am meisten riskiert.->k15-als-jamal]]  
[[Michael. Der die Fragen stellt.->k15-als-michael]]  
[[Kat. Die die Hubs kennt.->k15-als-kat]]  
[[Lena. Die es kommen sah.->k15-als-lena]]  
[[Tom. Der am wenigsten zu verlieren hat.->k15-als-tom]]
```

Danach folgt Ja oder Nein. Die Novelle verlangt Einstimmigkeit ("Wenn auch nur einer Nein sagt, ist das Projekt vorbei."). Ein Nein könnte die Geschichte an dieser Stelle beenden, und genau das soll es nicht, denn die Novelle bleibt kanonisch. Gelöst ist das nicht durch Ignorieren der Wahl, sondern indem die Gruppe ihre eigene Regel unter Druck neu verhandelt: die Figur des Spielers legt kein Veto gegen das Projekt ein, sondern gegen die eigene Beteiligung.

```
:: k15-nein [kapitel-15 teil-2]  
{ausgestiegen:1 ""}  
...  
{"" fuse (figur,": Ich sage nicht, dass ihr es lassen sollt. Ich sage, dass ich  
es nicht mittrage.")}  
...  
Am Ende blieben fünf. Einer ging, und nahm alles mit, was er wusste.  
  
[[Michael stand auf.->k15-konsequenzen]]
```

Fünf machen weiter, einer geht. **ausgestiegen** bleibt für den Rest der Geschichte gesetzt und färbt Schwur, Teil III und den Epilog, etwa im Schwur am Ende von Kapitel 15:

```
{if ausgestiegen "Fünf nickten. Der sechste Platz am Tisch war leer, und der  
Anhänger ging an ihm vorbei." else "Alle nickten." end}
```

Beide Wege (**k15-ja** und **k15-nein**) führen anschließend wieder in **k15-konsequenzen** und damit zurück in die Hauptlinie zusammen: es ist die einzige Stelle im ganzen Spiel, an der eine Spielerentscheidung den Bestand der Gruppe ändert, und sie kostet etwas, ohne die Handlung strukturell zu brechen.

7.5. Die Variablen im Überblick

Variable	Bereich	Gesetzt durch	Wirkt auf
entschlossenheit	0 bis 8	Haltungswahlen Sarah, Lena	Tonfall in K22, K24, Epilog

Variable	Bereich	Gesetzt durch	Wirkt auf
vorsicht	0 bis 8	Haltungswahlen Jamal, Michael	Schwere von K21 (Toms Fehler), K23
zusammenhalt	0 bis 8	Haltungswahlen Kat, Tom	Epilog, Schlussbild
figur	Name	K15	wessen Abstimmung der Spieler spricht
ausgestiegen	0 oder 1	K15	Varianten in Teil III und Teil IV
tom_gestanden	0 oder 1	K21	K23, K28 (Prozess)

Diese Variablen bleiben absichtlich wenige: mehr Variablen würden die Kombinatorik der Textvarianten schnell unübersichtlich machen. Ein zweiter Wendepunkt liegt in Kapitel 21: Tom entdeckt seinen eigenen Fehler und entscheidet, ihn zu gestehen oder zu verschweigen (**tom_gestanden**), was den Ton der Entdeckung in Kapitel 23 und den Prozess in Kapitel 28 verändert, ohne dass die Handlung selbst abzweigt.

Textvarianten aus diesen Variablen entstehen direkt im twee, wie in [Das Datenformat](#) gezeigt:

```
{if entschlossenheit>5 "Sie zögerte keine Sekunde." else "Sie zögerte." end}
```


Chapter 8. Bearbeiten und erweitern

Dieses Kapitel zeigt an konkreten Abläufen, wie du das Spiel veränderst. Für die meisten Änderungen brauchst du keine einzige Zeile Lil, nur einen Texteditor und die Befehle aus [Einrichtung](#).



Öffne `game/data/szenen.twee` und `game/data/emotes.csv` in einem einfachen Texteditor, nicht in Excel oder Numbers für die CSV-Dateien. Tabellenprogramme ändern gerne heimlich das Dateiformat, und dann funktioniert der Bau nicht mehr.

8.1. Text ändern

Suche die passende Passage in `game/data/szenen.twee` (der Passagenname verrät Kapitel und Ort, zum Beispiel `k07-vorschlag`), ändere den Text zwischen zwei Regieanweisungen oder Leerzeilen, speichere und baue neu:

```
lilt game/build/build_vn.lil # oder: make deck
```

Öffne danach `game/build/netzwerk.deck` in Decker neu, siehe [Das Spiel spielen](#).

8.2. Eine Szene ergänzen

Eine neue Passage braucht mindestens Kopfzeile, Regie, Text und einen Ausgang. Minimalbeispiel, angehängt an eine bestehende Passage:

```
:: k07-nachgedacht [kapitel-07 teil-1 pov-sarah]
!show sarah denk

Sarah dachte einen Moment nach, bevor sie antwortete.

[[Weiter.->k07-schluss]]
```

Verlinke die neue Passage von irgendwo aus (ein bestehender Fortsetzungslink oder eine neue Auswahl), sonst ist sie zwar vorhanden, aber unerreichbar. Baue neu; `build_vn.lil` meldet am Ende eine Zeile wie

```
Szenen: 166 Passagen, 1 Ende(n): k30-epilog-...
```

die auch unerreichbare Sackgassen mit auflistet, falls du eine Passage versehentlich ohne Ausgang lässt.

8.3. Eine Entscheidung einbauen

Aus einer Fortsetzung wird eine Entscheidung, sobald du einen zweiten Link ergänzt und eine **?**-Frage davor setzt:

```
? Was antwortet Sarah?  
  
[[Ja, sofort.->k07-ja]]  
[[Erst abwarten.->k07-abwarten]]
```

Beide Zielpassagen müssen existieren. Willst du damit auch eine der Haltungsvariablen beeinflussen, setze sie am Anfang der jeweiligen Zielpassage, mit abschließendem **"** (siehe [Das Datenformat](#)):

```
:: k07-ja [kapitel-07 teil-1 pov-sarah]  
{entschlossenheit:entschlossenheit+1 ""}  
...
```



Enthält die Beschriftung eines Links einen Bindestrich, verschwindet der Link spurlos aus dem Spiel (siehe [Das Datenformat](#)). Formuliere Beschriftungen notfalls um, Sprungziele dürfen Bindestriche unverändert enthalten.

8.4. Ein Emote hinzufügen

Ein siebtes Emote braucht drei Schritte:

1. Eine neue Zeile in `game/data/emotes.csv` mit allen Spalten (`emote,bow,braue_innen,braue_aussen,lid,pupille_x,pupille_y,mund_offen,augen_gross,bogen_daempfung`).
2. `tools/portraits.py` neu laufen lassen (`make portraits`), das zeichnet das neue Emote für alle sechs Figuren in allen drei Blickrichtungen und ergänzt `game/data/art.csv` automatisch.
3. `tools/render_art.py` (`make art`), das wandelt die neuen SVGs nach GIF um.

Bis das Emote gezeichnet ist, ist es im Spiel trotzdem benutzbar: `build_vn.lil` erzeugt für ein in `szenen.twee` referenziertes, aber noch nicht vorhandenes Emote automatisch einen Platzhalter aus `neutral` und meldet das als Hinweis, nicht als Fehler:

```
Platzhalter (Emote noch nicht gezeichnet, nutzt neutral):  
sarah/mutig
```

8.5. Einen Hintergrund ergänzen

Neue Orte entstehen in `tools/szenen.py`: eine neue Zeichenfunktion nach dem Muster der bestehenden (siehe [Die Bilder](#), Formenschatz aus Kasten, Tisch, Stuhl, Fenster und so weiter), eingetragen im **SZENEN**- Dictionary am Ende der Datei. Dann:

```
python3 tools/szenen.py <name>    # nur diesen einen Hintergrund
python3 tools/render_art.py hintergruende/<name>
```

Referenziere den Namen danach mit `!bg <name>` in der Szenendatei. Fehlt das Bild noch, bricht der Bau nicht ab, sondern warnt nur:

```
Warnungen:
  Hintergrund fehlt noch: <name>
```

Ein zum Test zusammengesetztes Beispielbild (Hintergrund plus Figuren) lässt sich jederzeit mit `tools/szenenbild.py` erzeugen, ohne das ganze Deck zu bauen:

```
python3 tools/szenenbild.py --ziel /tmp/probe.png <hintergrund> sarah:ent:links
michael:sorge:rechts
```

8.6. Was der Bau prüft, und was die Fehlermeldungen bedeuten

`build_vn.lil` prüft die Daten, bevor überhaupt ein Deck entsteht. Eine Auswahl der Fehlermeldungen und ihrer Ursache:

Meldung	Bedeutung
Startpassage "..." gibt es nicht	Der "start"-Wert im <code>StoryData</code> -Kopf von <code>szenen.twee</code> zeigt auf eine nicht existierende Passage.
Sprungziel fehlt: <von> → <ziel>	Ein <code>[[...→ziel]]</code> -Link (oder ein <code>rtext.make[...]</code>) zeigt auf eine Passage, die es nicht gibt. Meist ein Tippfehler im Passagennamen.
Bindestrich in Link-Beschriftung (Ply verliert den Link): <code>[[...]]</code> in ...	Siehe oben: Beschriftung umformulieren, ohne Bindestrich.
unbekannte Puppe: <name>	Ein <code>!show</code> , <code>!hide</code> , <code>!move</code> , <code>!flip</code> , <code>!order</code> , <code>!anim</code> oder <code>!talk</code> nennt einen Figurennamen, für den es keinen Ordner unter <code>game/data/art/figuren/</code> gibt. Meist ein Tippfehler im Puppennamen (er ist immer klein geschrieben, <code>sarah</code> nicht <code>Sarah</code>).

Meldung	Bedeutung
unbekannte Buehnenposition: <pos>	Die Position hinter !show oder !move ist keine der von Puppeteer bekannten Positionen. pt fällt bei einem unbekannten Namen sonst still auf bottom zurück, ein Tippfehler wäre also nur als leicht falsch stehende Figur sichtbar, deshalb prüft der Bau das explizit.
unbekanntes Kommando: !<name>	Ein ! -Kommando, das weder zu den eigenen (bg , zeit , hell , ...) noch zu Puppeteers Kommandos gehört. Meist ein Tippfehler.
Puppen ueberlappen in <passage>: <a> und (nur zwei Figuren passen nebeneinander, links und rechts)	Mehr als zwei sichtbare Puppen stehen in derselben Bühnenspalte. Auf die 1024 Pixel breite Bühne passen bei 480 Pixel Puppenbreite nur zwei Figuren gleichzeitig, eine links und eine rechts, siehe Software-Architektur . Löse Ensemble-Szenen über !hide und !show , nicht über mehr als zwei gleichzeitig sichtbare Figuren.
!bg ohne !clear in Passage <name> (Puppen der vorigen Szene blieben stehen)	Ein Szenenwechsel mit neuem Hintergrund, aber ohne vorheriges !clear . Ohne !clear bleiben die Puppen der vorigen Szene auf der Bühne stehen und überlagern sich mit den neuen.
Fehler im Lil-Fragment in <passage>: <meldung>{...}	Ein {lil} -Fragment lässt sich nicht auswerten, meist ein fehlendes end oder eine falsch platzierte Klammer. Ohne diese Prüfung würde Ply den Fehler stillschweigend schlucken und die Textstelle einfach verschwinden lassen, siehe Das Datenformat .
Skriptfehler in ...: ...	Ein von build_vn.lil erzeugtes Deck-, Karten- oder Widget-Skript lässt sich nicht parsen. Decker selbst würde das ignorieren und der betroffene Knopf würde einfach nichts tun; der Bau macht den Fehler stattdessen sichtbar.

Ein fehlender Hintergrund ist, wie oben gezeigt, nur eine Warnung, kein Fehler: das Deck bleibt spielbar, auch bevor alle Zeichnungen fertig sind.

8.7. Änderungen automatisch neu bauen

```
make watch
```

beobachtet `game/data`, `game/lil` und `game/build` und baut bei jeder Änderung automatisch Tests und Deck neu (braucht `entr`, `brew install entr`). Das ersetzt nicht das erneute Öffnen in Decker, siehe [Das Spiel spielen](#).

Chapter 9. Lil für Ungeübte

Dieses Kapitel gibt dir gerade so viel Lil, wie du zum Lesen und vorsichtigen Ändern von `game/lil/vn.lil` und `game/build/build_vn.lil` brauchst. Für alles Weitere: ["Learn Lil in 10 Minutes"](#) und die [Kurzreferenz](#).

9.1. Werte und Variablen

Ein Doppelpunkt weist einer Variable einen Wert zu, nicht das Gleichheitszeichen:

```
alter:13
name:"Sarah"
```

9.2. Funktionen aufrufen und definieren

Funktionsaufrufe benutzen eckige Klammern, Argumente stehen ohne Kommas nebeneinander. `on ... do ... end` definiert eine Funktion, ihr letzter Ausdruck ist automatisch ihr Rückgabewert, ein eigenes `return`-Wort wie in anderen Sprachen gibt es nicht:

```
on vn_rest x wort do
  n:1+count wort
  n drop x
end
```

9.3. Bedingungen und Schleifen

```
if alter>12
  show["du darfst mitspielen"]
end

each ding in "Schwert","Schild","Trank"
  show[ding]
end
```

9.4. Die wichtigste Falle: Lil hat keine Rechenregeln

In den meisten Programmiersprachen gilt "Punkt vor Strich". In Lil gibt es diese Regel **nicht**. Alles wird strikt von rechts nach links ausgewertet, egal welche Operatoren beteiligt sind:

```
show[10-4/2]
// ergibt 8, nicht wie erwartet 18
```



Deshalb gilt im ganzen Projekt eine feste Regel: sobald mehr als ein Rechenzeichen in einem Ausdruck vorkommt, wird geklammert, und jede Berechnung landet zuerst in einer eigenen Variable, bevor sie in ein Tupel oder eine Liste einfließt. `(px+m,py+m)` ist zum Beispiel **nicht** das Paar `(px+m, py+m)`, sondern `px+(m, (py+m))`. Richtig:

```
x0:px+m
y0:py+m
(x0,y0)
```

9.5. Zehn Fallstricke, alle tatsächlich beim Bau dieses Projekts aufgetreten

Diese Sammlung stammt aus `zelda-network/game/README.md` (Abschnitt "Sechs Lil-Fallstricke") und aus `plans/graphic-novel-plan.md`, Abschnitt 13. Beide gelten unverändert für dieses Projekt.

1. In **where-Klauseln gehört = hin, nicht ~**. `where spalte~"x"` ist ein stiller No-Op-Filter, `where spalte="x"` filtert richtig. In der Engine hätte die naheliegende, aber falsche Form `where 0<count arg` sogar bei richtigem Vergleichsoperator nicht funktioniert, weil `count` eine ganze Spalte aggregiert statt zeilenweise zu zählen, siehe [Software-Architektur](#).
2. **each v k i in dict liefert zuerst den WERT, nicht den Schlüssel**. Beim Durchlaufen von `deck.cards` oder `card.widgets` führt die falsche Reihenfolge zu leeren Namen und einer stillen Fehlsuche.
3. **Funktionsaufrufe vor einem Komma in einer Liste immer klammern**. Ohne Klammern verschluckt der Aufruf sonst den Rest der Liste als eigenes Argument, ohne Fehlermeldung.
4. **Verkettung ist "" fuse (a,b,c), nicht "prefix" fuse (a,b)**. `fuse` benutzt sein linkes Argument als Trennzeichen **zwischen** den Elementen rechts. `"prefix" fuse (a,b)` ergibt also `a+"prefix"+b`, bei einem zusammengesetzten Pfad einen gültigen, aber falschen String.
5. **Ein Modul-.value, das mit einer nackten Tabelle endet, wird zu einem flachen Spalten-Dict gecastet**. Tabelle vorher in ein Dict-Feld einbetten.
6. **Zuweisungen in einer Funktion sind NICHT automatisch lokal**. Ohne `local` schreibt `x:...` in den umgebenden Gültigkeitsbereich, nicht in einen neuen, funktionslokalen. `vn_ohne_rand` in `game/lil/vn.lil` zeigt deshalb explizit, wie man es richtig macht:

```
# Fuehrende Leerzeichen und Zeilenumbrueche abschneiden.
# ACHTUNG: Zuweisungen in einer Lil-Funktion sind NICHT automatisch lokal.
# Ohne "local" schreibt "s:..." in den umgebenden Gueltigkeitsbereich und
# wuerde hier den Spielstand von vn_spiele ueberschreiben, der ebenfalls "s"
# heisst.
on vn_ohne_rand t do
  local s:t
  local weiter:1
  while weiter
    if 0~count s
      weiter:0
    elseif (first s) in (" ", "\n")
      s:1 drop s
    else
      weiter:0
    end
  end
end
s
end
```

In `build_vn.lil` traf genau dieser Fehler einmal die Prüffunktion selbst: eine Hilfsvariable ohne `local` überschrieb dort die globale Liste aller Passagennamen, woraufhin der Bau plötzlich **jedes** Sprungziel als fehlend meldete. Das Symptom zeigte dabei nicht im Entferntesten auf die Ursache. Hilfsvariablen in Funktionen deshalb immer mit `local` deklarieren.

7. `by` ist ein reserviertes Wort und als Variablenname nicht erlaubt. `vn_heim_geklickt` in `game/lil/vn.lil` benennt deshalb, was sonst naheliegend `bx/by` hieße, um:

```
# "by" ist in Lil ein reserviertes Wort und als Variablenname
# nicht erlaubt.
local kx:first b.pos
local ky:last b.pos
local krechts:kx+first b.size
local kunte:ky+last b.size
```

Reservierte Wörter lassen sich auch nicht als Funktionsparameter benutzen, und die Fehlermeldung dabei ist nicht immer eindeutig (`format` als Parametername ergibt zum Beispiel `'format' is a keyword...`, `table` als lokale Variable ein rätselhaftes `Expected name, but found :`). Im Zweifel kurz mit `lilt -e 'wort:1 show[word]'` testen, ob ein Wort als Variablenname geht.

8. Ein fehlerhaftes Lil-Fragment in der Szenendatei verschwindet spurlos. Ply führt `{...}`-Fragmente aus und schluckt Fehler dabei stillschweigend. Siehe [Das Datenformat](#) für die Auswechlösung (`eval[]` mit Fehlerfeld `error`).

9. `readcsv[]` auf einer kopfzeilenlosen Zahlen-CSV kann `lilt` segfaulten lassen (Exit 139). In diesem Projekt weniger relevant, weil `game/data/emotes.csv` und `game/data/art.csv`

eine Kopfzeile tragen und ohnehin per Python gelesen werden, nicht per Lil.

10. `x.paste[bild pos]` behandelt Pattern 0 nicht automatisch als transparent. Dafür ein drittes, wahres Argument nötig: `x.paste[bild pos 1]`. Das benutzt zum Beispiel `vn_hintergrund` beim Kopieren des Bühnenbilds nicht, weil Hintergründe deckend sind, wohl aber Puppeteer intern beim Zeichnen der Puppen-Canvases.

9.6. Ein echtes Beispiel: Sprechererkennung

So liest `vn_sprecher` in `game/lil/vn.lil` aus einem Textabsatz heraus, ob und wer spricht:

```
on vn_sprecher roh do
  r:()
  r.name:""
  r.ab:0
  treffer:rtext.find[roh ":"]
  if 0<count treffer
    ende:first first treffer
    kopf:vn_ohne_rand[ende take roh]
    gut:1
    if 0~count kopf gut:0 end
    if 24<count kopf gut:0 end
    if 0<count rtext.find[kopf "\n"] gut:0 end
    woerter:extract where count@value from " " split kopf
    if 3<count woerter gut:0 end
    each w in woerter
      if !(first w) in GROSS gut:0 end
    end
    if gut
      r.name:kopf
      r.ab:1+ende
    end
  end
end
r
end
```

Das zeigt mehrere Bausteine auf einmal: `rtext.find[]` sucht eine Teilzeichenkette, `first first treffer` holt die Position des ersten Treffers, `extract where count@value from ...` `split kopf` filtert eine Liste über eine Bedingung auf jedem Element, und ein Dictionary wird mit `r:()` feldweise aufgebaut statt in einem Ausdruck, genau damit die Rechtsassoziativität aus Fallstrick 3 keine Überraschung erzeugt.

9.7. Zum Weiterlesen

- ["Learn Lil in 10 Minutes"](#)
- [Lil-Kurzreferenz](#)
- [Das vollständige Lil-Handbuch](#)

- [plans/graphic-novel-plan.md](#), Abschnitt 13, für die ausführliche Fassung dieser Fallstricke mit Fundgeschichte.
- [../zelda-network/game/README.md](#), Abschnitt "Sechs Lil-Fallstricke", die Quelle der ersten sechs Punkte oben.

Chapter 10. Glossar

Kurze Erklärungen der Fachwörter aus diesem Handbuch, in der Reihenfolge des Alphabets.

AsciiDoc

Ein einfaches Textformat, aus dem sich Bücher, Webseiten und PDFs erzeugen lassen. Dieses Handbuch selbst ist in AsciiDoc geschrieben.

Contraption

Englisch für "Apparat" oder "Vorrichtung". In Decker eine Sammlung von Widgets, die zusammen etwas Wiederverwendbares ergeben, wie ein eigener kleiner Baustein. `dd` und `pt` sind selbst als solche Bausteine gebaut.

Deck

Eine ganze Decker-Datei, zum Beispiel `game/build/netzwerk.deck`. Vergleichbar mit einem Karteikasten voller Karten.

Dialogizer (`dd`)

Das Decker-Modul für die Präsentationsschicht: modale Dialogbox, Klick-durch-Text, Auswahlknöpfe, Textanimation. Siehe [Software-Architektur](#).

Docker

Ein Werkzeug, mit dem Programme in einer abgeschlossenen, vorbereiteten Umgebung laufen, ohne dass man sie selbst installieren muss. Wird hier benutzt, um dieses Handbuch aus AsciiDoc zu bauen, siehe `handbuch/README.md`.

Emote

Ein Gesichtsausdruck einer Figur, zum Beispiel `neutral`, `sorge` oder `schock`. Steht als Zeile in `game/data/emotes.csv` und als eigenes Canvas-Widget auf dem Figurenblatt jeder Puppe. Siehe [Die Bilder](#).

Fediverse

Ein Verbund unabhängiger, aber über offene Protokolle (etwa ActivityPub) miteinander verbundener sozialer Netzwerke, zum Beispiel Mastodon oder PeerTube. In der Novelle das technische Vorbild für das föderierte Sendernetzwerk, das die Figuren aufbauen.

Haltungswahl

Eine der häufigen, ein bis zwei Passagen später wieder zusammenlaufenden Entscheidungen in der Geschichte. Verändert eine Haltungsvariable (`entschlossenheit`, `vorsicht`, `zusammenhalt`) und damit späteren Text, nicht aber den Handlungsverlauf selbst. Siehe [Die Handlung](#).

Hub

Ein autonomer Knoten des Sendenetzwerks in der Novelle, zum Beispiel der `nairobi-hub`. Nicht zu verwechseln mit einem Hintergrundbild, auch wenn ein Hub im Spiel als Hintergrund gezeigt wird.

Karte (in Decker)

Eine einzelne Seite oder Szene innerhalb eines Decks, vergleichbar mit einer Karteikarte. Siehe [Software-Architektur](#).

Lil

Die kleine Programmiersprache, die in Decker eingebaut ist. Siehe [Lil für Ungeübte](#).

Modul

Ein fertiger, in sich abgeschlossener Decker-Baustein mit eigenen Funktionen, hier `twee`, `dd` und `pt`. Siehe [Software-Architektur](#).

Passage

Eine Szene an einem Ort in `game/data/szenen.twee`: Text, Regie und Auswahlmöglichkeiten unter einem eindeutigen Namen. Siehe [Das Datenformat](#).

Ply

Das Twee-3-Story-Format, in dem `game/data/szenen.twee` geschrieben ist. Rendert Passagentext samt eingebetteter Lil-Fragmente zu einer rtext-Tabelle mit den Spalten `text`, `font`, `arg` und `pat`. Siehe [Software-Architektur](#).

Puppe

Die Bühnendarstellung einer Figur in Puppeteer: eine Karte, deren Name der Puppenname ist, mit einem Canvas-Widget je Emote. Jede Figur hat drei Puppen (`frontal`, `_r`, `_l`) für die drei Blickrichtungen. Siehe [Die Bilder](#).

Puppeteer (pt)

Das Decker-Modul für die Darstellungsschicht: Figuren auf der Bühne zeigen, bewegen, blinzeln, sprechen lassen. Siehe [Software-Architektur](#).

rtext

Deckers Rich-Text-Datentyp: eine Tabelle aus Textsegmenten mit Formatierung. `twee.render[]` liefert eine solche Tabelle als Ergebnis einer gerenderten Passage.

Twee

Ein einfaches Textformat für interaktive Fiktion (`:: Passagenname [tags] {metadaten}`, gefolgt von Text und `)`, ursprünglich aus dem Werkzeug Twine. `game/data/szenen.twee` ist im Story-Format Ply geschrieben. Siehe [Das Datenformat](#).

Widget

Ein einzelner Baustein auf einer Karte, zum Beispiel ein Knopf, ein Feld oder eine Zeichenfläche (Canvas). Siehe [Software-Architektur](#).

Impressum

Das Netzwerk: Handbuch zur Visual Novel

Autor: Olav Schettler

Kontakt: olav@schettler.net

Die Novelle "Das Netzwerk" und ihre spielbare Adaption sind vollständig fiktional. Alle Charaktere, Namen, Institutionen, Handlungen und Ereignisse sind frei erfunden; Ähnlichkeiten mit realen Personen oder Institutionen sind rein zufällig. Der ausführliche rechtliche Hinweis steht im Spiel selbst unter dem Knopf "Rechtlicher Hinweis" auf der Titelfarte ([game/data/hinweis.txt](#)).

Dieses Werk steht unter der Lizenz [CC BY-SA 4.0](#). Du darfst es teilen und verändern, solange du den Autor nennst und Bearbeitungen unter derselben Lizenz weitergibst. Die Novelle selbst steht unter [CC BY 4.0](#).

Geschrieben mit AsciiDoc, gebaut mit AsciiDoctor.